



GlobalLogic®

AWS Lambda: Features and Uses

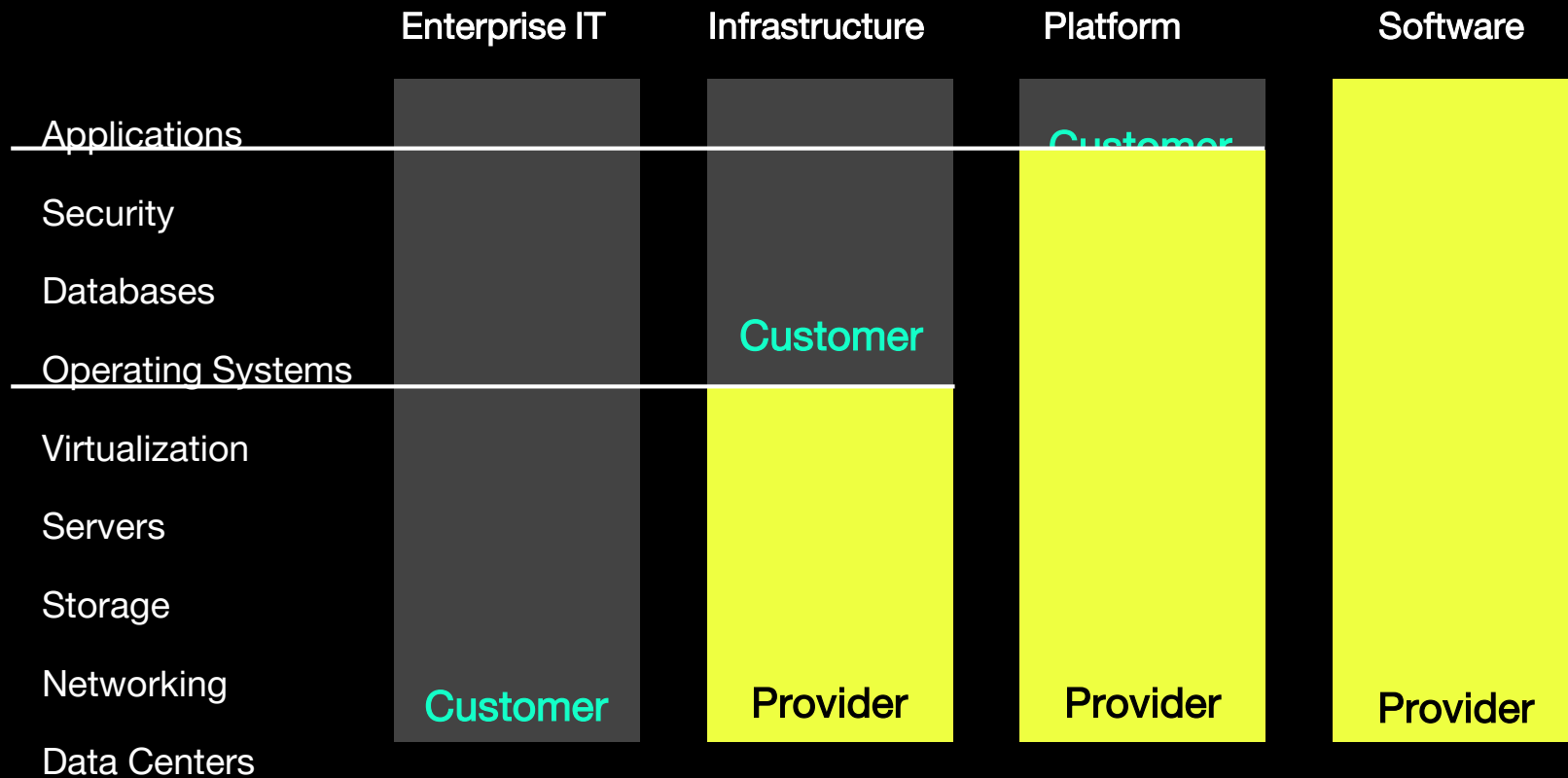
Dmytro Harbuzov

Senior Software Engineer, Consultant

Agenda

1. Introduction
2. Amazon Web Services overview
3. AWS Lambda serverless computing
4. AWS Lambda pros and cons

Cloud computing



Amazon Web Services Overview

AWS product's categories



Compute



Storage



Database



Migration



Networking & Content
Delivery



Developer Tools



Management Tools



Media Services



Security, Identity &
Compliance



Analytics



Machine Learning



Mobile Services



AR & VR



Application Integration



Customer Engagement



Business Productivity



Desktop & App Streaming



Internet of Things



Game Development

Global Infrastructure



Global Infrastructure



AWS Lambda & serverless computing



What is serverless computing?

10

- SCALE: flexible scale without worrying about complex and time-consuming data migrations.
- SIMPLICITY: you don't manage any servers and OS.
- AVAILABILITY: managed by cloud provider
- COST: you pay only for the requests served and the compute time required to run your code



Supported services



Amazon data stores

S3
Dynamo DB
Kinesis
Cognito



End points

Amazon Alexa
Amazon API Gateway
AWS IoT



Repositories

AWS:

- CloudFormation
- CloudTrail
- CodeCommit

Amazon CodeCommit



Event/message services

Amazon SES
Amazon SNS
Cron events

Supported languages



Serverless applications

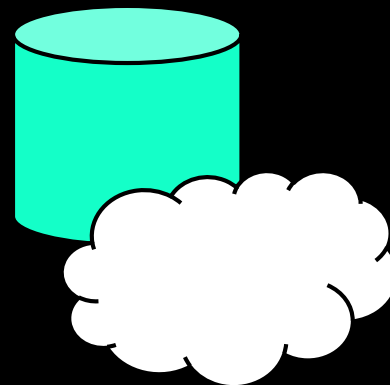
Event Source



Function



Services



Example synchronous model

Custom App



AWS Lambda



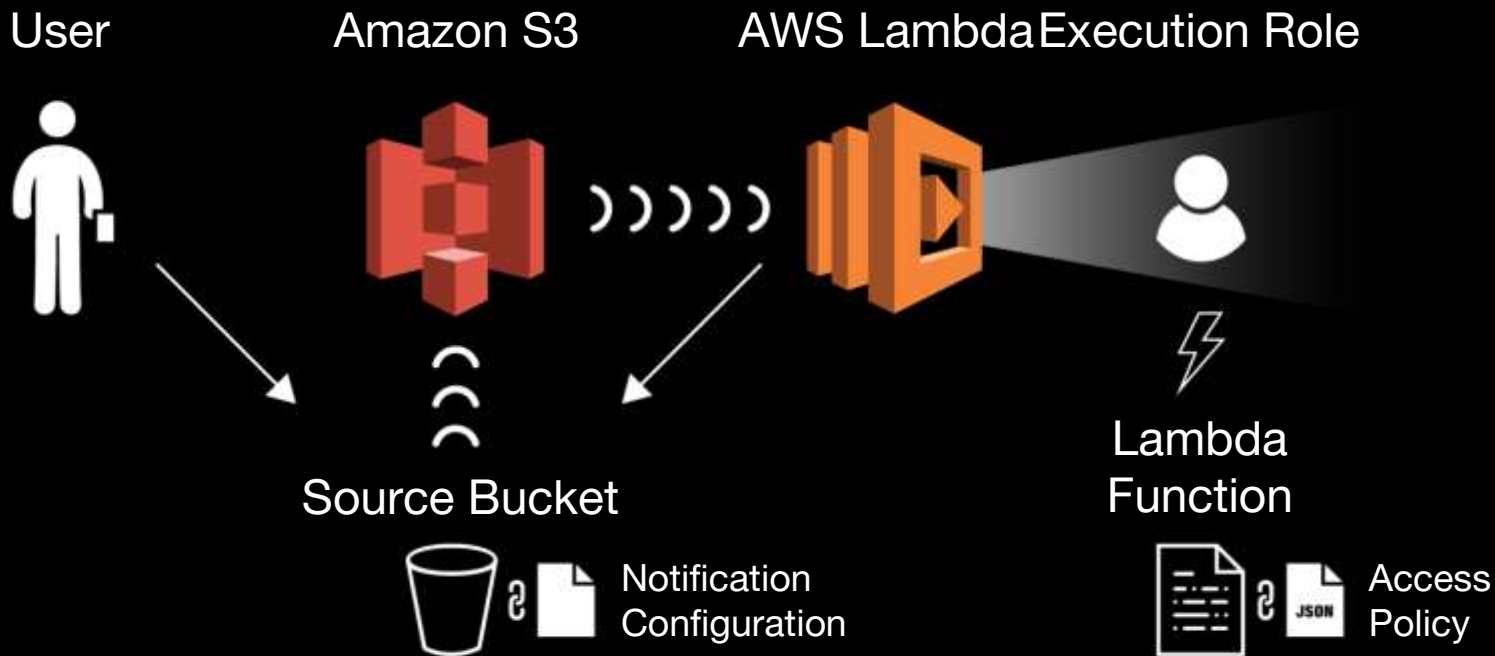
Execution Role



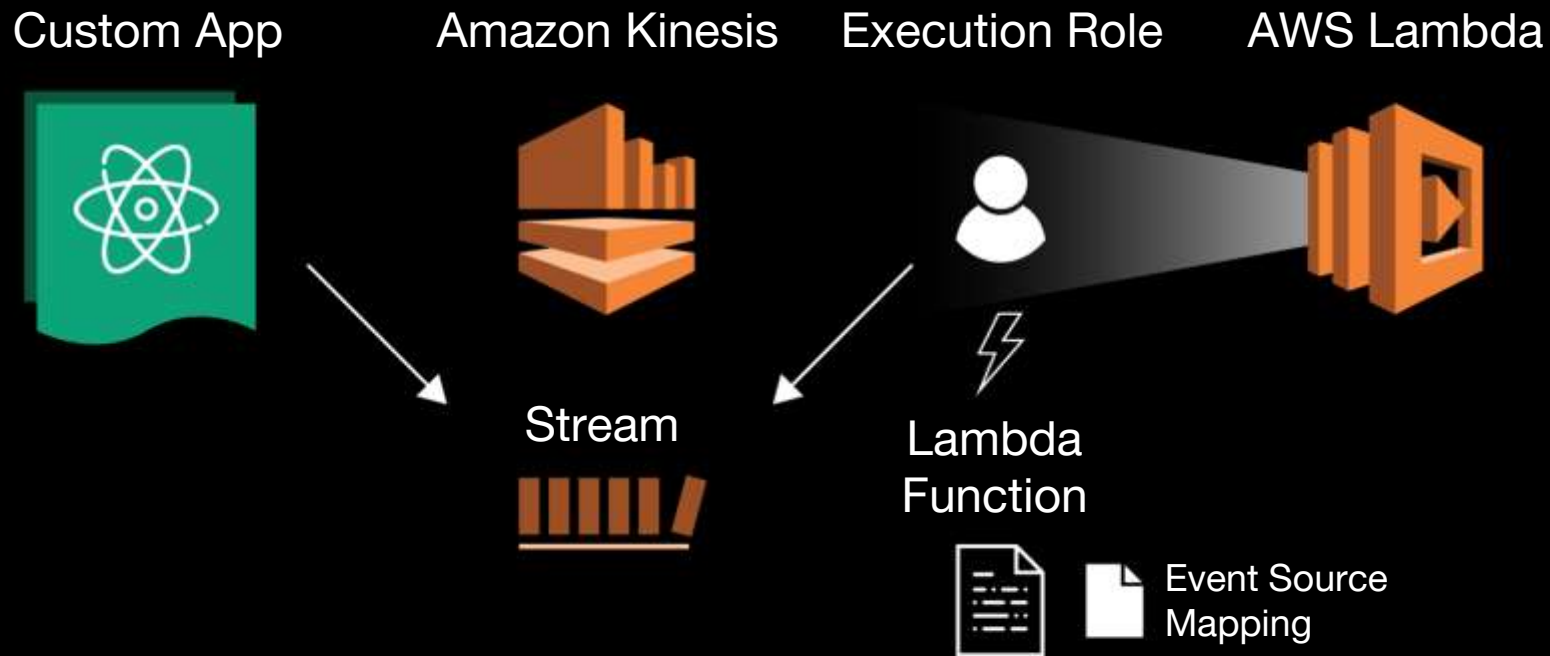
Lambda
Function



Example asynchronous model



Example stream-based model



Lambda permissions model

IAM Role

- Permissions you grant to this role determine what your AWS Lambda function can do
- If event source is Amazon DynamoDB or Amazon Kinesis, then add read permissions in IAM Role

Function (resource) policy

- Permissions you grant to your Lambda function determine which service or event source can invoke your function
- Resource policies make it easy to grant cross-account permissions to invoke Lambda function

Lambda Configuration

- Compute resources
- Maximum execution time (timeout)
- IAM role (execution role)
- Handler name

Lambda execution context

How?

Each Execution Context provides 500MB of additional disk space in the /tmp directory. The directory content remains when the Execution Context is frozen, providing transient cache that can be used for multiple invocations

What?

Any declarations in your Lambda function code (outside the handler code) remains initialized, providing additional optimization when the function is invoked again (for example Database connections, Environment Variables)

Programming model

- **Handler**. Handler is the function AWS Lambda calls to start execution of your Lambda function.
- **Context**. AWS Lambda also passes a `context` object to the handler function, as the second parameter. Via this context object your code can interact with AWS Lambda.
- **Logging**. Your Lambda function can contain logging statements. AWS Lambda writes these logs to CloudWatch Logs.
- **Exceptions**. Your Lambda function needs to communicate the result of the function execution to AWS Lambda.

Lambda Function Handler

```
outputType handler-name(inputType input, Context context) {  
    ...  
}
```

- **inputType**: The first handler parameter is the input to the handler, which can be event data (published by an event source) or custom input that you provide such as a string or any custom data object.
- **outputType**: If you plan to invoke the Lambda function synchronously (using the `RequestResponse` invocation type), you can return the output of your function using any of the supported data types. If you plan to invoke the Lambda function asynchronously (using the `Event` invocation type), the `outputType` should be `void`.

Lambda Context Object

- `getMemoryLimitInMB()`
- `getFunctionName()`
- `getFunctionVersion()`
- `getInvokedFunctionArn()`
- `getAwsRequestId()`
- `getLogStreamName()`
- `getLogGroupName()`
- `getClientContext()`
- `getIdentity()`
- `getRemainingTimeInMillis()`
- `getLogger()`

Lambda logging



AWS CloudWatch

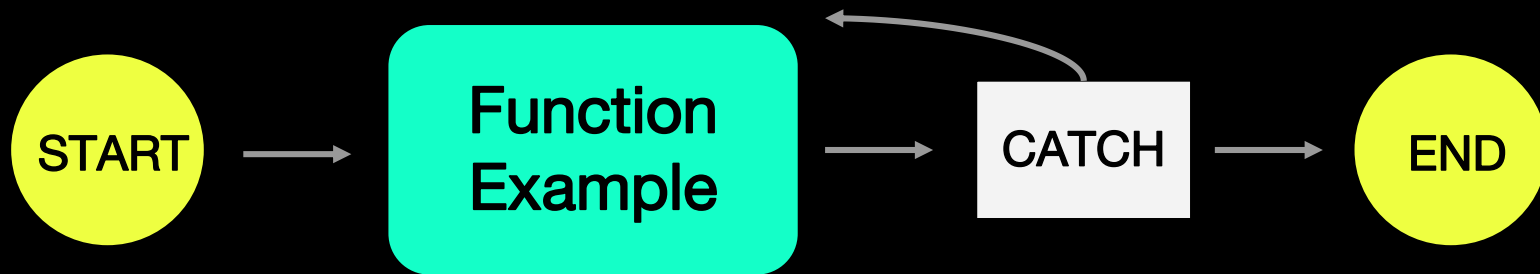
Options

- AWS Lambda recommends Log4j 2 to provide a custom appender. You can use the custom Log4j appender provided by Lambda for logging from your lambda functions.
- In addition, you can also use the statements “System.out()” and “System.err()” in your Lambda function code to generate log entries.

Lambda error handling

```
{
  "errorMessage": "Name John Doe is invalid. Exception occurred...",
  "errorType": "java.lang.Exception",
  "stackTrace": [
    "example.Hello.handler(Hello.java:9)",
    "sun.reflect.NativeMethodAccessorImpl.invoke0(Native Method)",
    "sun.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:62)",
    "sun.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43)",
    "java.lang.reflect.Method.invoke(Method.java:497)"
  ]
}
```

Lambda error handling with AWS Step Functions



Limits per invocation

Resource

Limits

Memory allocation range	Min 128 MB / Max 3008 MB (with 64 MB Increments)
Ephemeral disk capacity (* /tmp* space)	512 MB
Number of file descriptors	1,024
Number of processes and threads (combined total)	1,024
Max execution duration per request	300 seconds
Invoke request body payload size (RequestResponse/synchronous invocation)	6 MB
Invoke request body payload size	128 KB

Deployment limits

Item	Default Limit
Lambda function deployment package size (compressed .zip/.jar file)	50 MB
Total size of all the deployment packages that can be uploaded per region	75 MB
Size of code/dependencies that you can zip into a deployment package (uncompressed .zip/.jar file)	250 MB
Note - Each Lambda function receives an additional 500 MB of non-persistent disk space in it's own /tmp directory. The /tmp directory can be used for loading additional resources like dependency libraries or data or data sets during function initialization	
Total size of environment variables set	4 KB

Lambda pricing

Free Tier

1 M requests
per month

400,000 GB-SECONDS
of compute time per month

The Lambda free tier does not automatically expire at the end of your 12 month AWS Free Tier term, but is available to both existing and new AWS customers indefinitely.

Requests

1 M requests free
First 1M requests per month
are free

\$0.20 Per 1M requests
thereafter
\$0.0000002 per request

Duration

400,000 GB-SECONDS per
month free
First 400,000 GB-seconds
per month up to 3.2M
seconds of compute time, are
free

\$0.00001667 for every GB-
second used thereafter
The price depends on the
amount of memory you
allocate to your function

Lambda pros and cons



- Scaling
- Faster Development
- Operational Management
- Reduction in Operational
- Costs

Lambda pros and cons



- Scaling
- Faster Development
- Operational Management
- Reduction in Operational
- Costs



- Execution Duration
- Stateless
- Testing
- Monitoring and Debugging
- Denial of Service
- Tracking



Thank you!

dmytro.harbuzov@gmail.com

GlobalLogic